Previsao de demanda e otimizacao de estoque: por que este problema importa

Resumo. *Este primeiro artigo abre a serie com o problema de negocio que vai sustentar todos os experimentos seguintes: previsao de demanda para reposicao de estoque. O objetivo e mostrar por que esse problema e valioso, por que ele e dificil e por que ele cria um caminho didatico natural ate Reservoir Computing (RC) e Quantum Reservoir Computing (QRC). Em vez de comecar pela teoria do reservatorio, comecamos pelo custo do erro no varejo, formalizamos a tarefa de previsao, fixamos o recorte experimental no dataset Favorita e mostramos as primeiras estatisticas reais da serie* `store_nbr = 1` *e* `family = BEVERAGES`*. Ao final, o leitor sabe como carregar o recorte, entende as metricas que vao guiar a serie, enxerga o impacto de promocao e calendario e tem um mapa completo dos sete artigos.*

0.1. 1. O que o leitor vai aprender

Ao final deste artigo, voce sera capaz de:

1. explicar por que previsao de demanda e um problema central de negocio;
2. formular matematicamente a tarefa de previsao da serie;
3. reproduzir o recorte adotado no Favorita em toda a obra;
4. interpretar os primeiros artefatos computacionais antes de treinar qualquer modelo;
5. entender por que esse problema e uma boa rota pedagogica ate QRC.

0.2. 2. O problema de negocio antes do modelo

Previsao de demanda e importante porque o erro entra diretamente na decisao de estoque. Se a empresa compra menos do que deveria, perde venda. Se compra mais, imobiliza capital e aumenta o risco de desperdicio. Em um ambiente de varejo alimentar, isso acontece em escala diaria, com milhares de decisoes pequenas produzindo efeitos acumulados grandes.

Um modo simples de formalizar esse efeito e escrever o erro de previsao como

$$e_{t+h} = y_{t+h} - \hat{y}_{t+h},$$

em que y_{t+h} e a demanda observada e $\hat{y}_{t+h}$ e a previsao produzida no tempo t para o horizonte h.

Esse erro alimenta uma politica operacional minima de reposicao:

$$R_t = \hat{y}_{t+1} + SS_t,$$

em que R_t e a quantidade planejada para reposicao e SS_t e um estoque de seguranca. Mesmo sem modelar toda a politica de inventario, a equacao deixa claro o ponto principal: melhorar $\hat{y}_{t+1}$ melhora a qualidade da decisao.

0.3. 3. Da pergunta de negocio para a tarefa de previsao

Ao longo da serie, vamos modelar a tarefa como

$$\hat{y}_{t+h} = f(y_{1:t}, x_{1:t+h}),$$

em que:

- $y_{1:t}$ representa o historico de vendas observado ate o tempo t;
- $x_{1:t+h}$ representa sinais exogenos, como promocao e calendario;
- $f(\cdot)$ sera implementada por modelos estatisticos, de IA classica, RC e QRC.

Essa forma e importante pedagogicamente porque nos permite manter o mesmo problema enquanto trocamos a familia de modelo. O que muda do artigo 1 ao 7 nao e a pergunta. O que muda e a maneira de construir a representacao temporal.

0.4. 4. O dataset Favorita e o recorte didatico da serie

O projeto usa o dataset da competicao *Corporacion Favorita Grocery Sales Forecasting* como fio condutor. Para manter o ensino progressivo e reproduzivel, toda a serie comeca no mesmo recorte:

- loja: `store_nbr = 1`
- familia: `BEVERAGES`
- frequencia: diaria
- teste: ultimos `90` dias

A tabela abaixo resume o recorte real usado nas implementacoes.

Metrica	Valor
Dias totais no recorte	1688
Dias de treino	1598
Dias de teste	90
Media diaria de vendas	1583.99
Desvio padrao das vendas	729.89
Taxa media de promocao	60.37%
Taxa de dias com venda zero	0.59%
Inicio do treino	2013-01-01
Inicio do teste	2017-05-18
Fim do teste	2017-08-15

O carregamento desse recorte e implementado em `code/common/favorita.py`. O nucleo da reproducao e o seguinte:

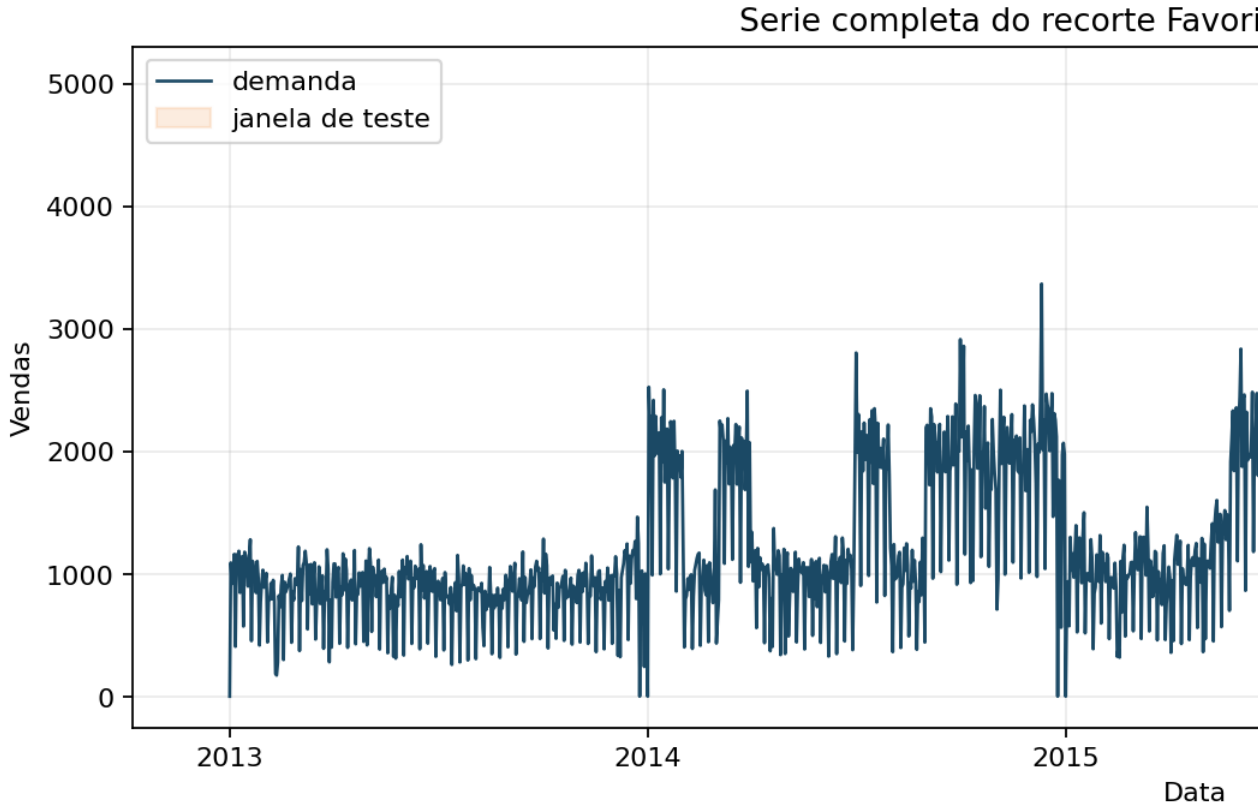
```
from common.favorita import (
    FavoritaSeriesConfig,
    load_store_family_series,
    temporal_train_test_split,
)

config = FavoritaSeriesConfig(store_nbr=1, family="BEVERAGES", test_day
frame = load_store_family_series(store_nbr=config.store_nbr, family=con
train, test = temporal_train_test_split(frame, test_days=config.test_da
```

Esse codigo faz tres coisas que sao centrais para toda a serie:

1. fixa um recorte comum para todos os modelos;
2. densifica o calendario diario, preenchendo dias faltantes com zero;
3. garante um split temporal identico para comparacoes posteriores.

0.5. 5. O que os dados ja ensinam antes de qualquer modelo

Antes de falar de RC ou QRC, vale observar o comportamento bruto da serie.

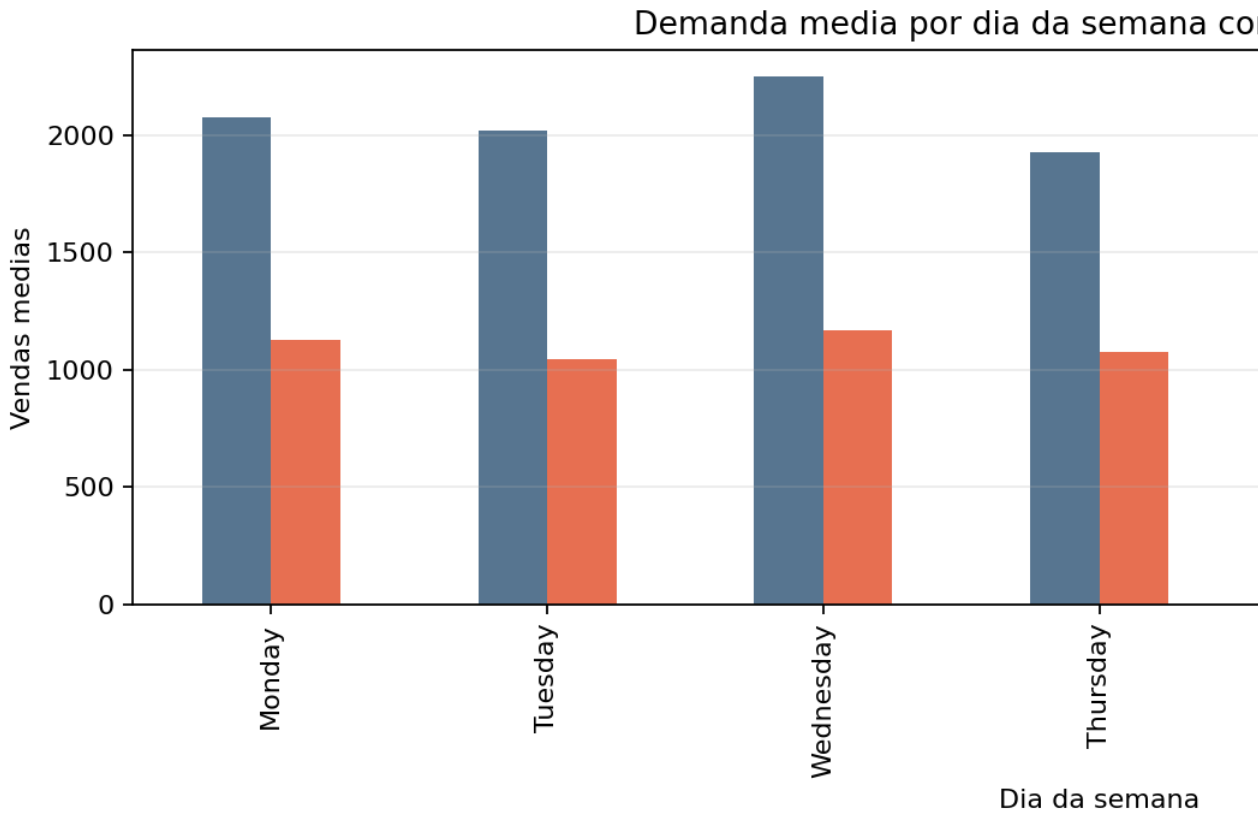


A visao geral mostra que o problema nao e trivial por pelo menos tres motivos:

- ha variacao forte ao longo do tempo;
- a serie responde a padroes semanais;
- promocao altera o patamar das vendas.

O perfil semanal medio confirma esse ultimo ponto.

Dia da semana	Media com promocao	Media sem promocao
Monday	2074.45	1125.28
Tuesday	2016.35	1046.04
Wednesday	2248.32	1166.31
Thursday	1927.36	1074.45
Friday	2116.13	1133.84
Saturday	2209.90	1232.53
Sunday	977.32	498.80



Duas leituras didaticas aparecem imediatamente:

1. a serie tem sazonalidade semanal clara, o que justifica baselines como `Seasonal Naive` e `ETS`;
2. promocao desloca a media de vendas em todos os dias da semana, o que torna natural incluir variaveis exogenas.

Esse e o primeiro motivo para escolher esse problema como rota ate QRC: mesmo o recorte inicial ja exige memoria temporal e sensibilidade a contexto.

0.6. 6. Passo-a-passo para reproduzir o recorte

O leitor pode reproduzir o ponto de partida da obra em quatro passos.

0.6.1. 6.1 Passo 1: localizar os dados

Os arquivos brutos foram organizados em `dataset/raw/`, com `train.csv`, `transactions.csv`, `stores.csv`, `holidays_events.csv`, `oil.csv` e `test.csv`.

0.6.2. 6.2 Passo 2: carregar uma unica serie

A funcao `load_store_family_series()` filtra por loja e familia, ordena por data e produz um `DataFrame` com as colunas:

- `ds`: data
- `y`: demanda diaria
- `onpromotion`: sinal exogeno basico usado nos modelos

0.6.3. 6.3 Passo 3: fixar o split temporal

A funcao `temporal_train_test_split()` implementa a regra que vamos herdar em toda a serie:

- o passado vai para treino;
- os ultimos `90` dias vao para teste;
- nenhuma observacao do futuro entra no treino.

0.6.4. 6.4 Passo 4: inspecionar o recorte

Os artefatos computacionais deste artigo ja mostram o minimo necessario para nao modelar no escuro:

- `series_overview.png`
- `weekly_profile.png`
- `recorte_summary.csv`

Em um projeto real, essa etapa nao e opcional. Ela evita treinar modelos sofisticados sobre uma serie mal definida.

0.7. 7. Como vamos medir desempenho

Toda a obra usa as mesmas quatro metricas, implementadas em `code/common/metrics.py`.

O erro absoluto medio e dado por

$$\mathrm{MAE} = \frac{1}{n} \sum_{t=1}^{n} |y_t - \hat{y}_t|.$$

O erro quadratico medio com raiz e dado por

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}.$$

O erro absoluto ponderado pelo volume da serie e dado por

$$\mathrm{WAPE} = \frac{\sum_{t=1}^{n}|y_t - \hat{y}_t|}{\sum_{t=1}^{n}|y_t|}.$$

E o erro percentual absoluto medio simetrico e dado por

$$\mathrm{sMAPE} = \frac{1}{n}\sum_{t=1}^{n}\frac{2|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}.$$

Essas metricas vao aparecer repetidamente porque cada uma responde a uma pergunta diferente:

- `MAE`: erro medio em unidades de venda;
- `RMSE`: sensibilidade a erros grandes;
- `WAPE`: erro relativo ao volume total;
- `sMAPE`: comparacao percentual mais estavel em series com escalas diferentes.

0.8. 8. Por que esse problema conduz naturalmente ate RC e QRC

Ha pelo menos quatro motivos.

1. A serie depende do passado, entao memoria importa.
2. O efeito de promocao e calendario nao e constante, entao nao linearidade importa.
3. O protocolo de negocio e simples de explicar, entao o leitor nao se perde no dominio.
4. O mesmo recorte pode ser usado para modelos estatisticos, IA classica, RC e QRC.

Em outras palavras, o problema e suficientemente real para ser relevante e suficientemente controlado para ser didatico.

0.9. 9. Roadmap dos sete artigos

O percurso da serie sera o seguinte:

1. artigo 1: problema de negocio, recorte e metricas;
2. artigo 2: fundamentos de Reservoir Computing;
3. artigo 3: primeiro pipeline reproduzivel com baseline e RC;
4. artigo 4: memoria, nao linearidade e tuning do RC;
5. artigo 5: benchmark justo entre todas as implementacoes;
6. artigo 6: ponte de RC classico para QRC;
7. artigo 7: estudo final de QRC no Favorita.

Essa organizacao evita um erro comum em textos de QRC: apresentar o formalismo quantico antes de o leitor entender a tarefa, o protocolo experimental e a logica do reservatorio.

0.10. 10. Conclusao

O ponto de partida da serie esta estabelecido. Temos um problema de negocio real, um recorte experimental fixo, um pipeline minimo de carregamento de dados e um conjunto de metricas que permitira comparacoes honestas. Esse chao comum e o que torna possivel ensinar RC e QRC sem perder o leitor em abstracoes.

O proximo artigo parte daqui para responder a pergunta central que o leitor provavelmente ja esta fazendo: afinal, o que e um reservatorio e por que ele pode ser util para uma serie temporal como essa?

0.11. Entregaveis associados no repositorio

- dados brutos: `dataset/raw/`
- carga do recorte: `code/common/favorita.py`
- metricas: `code/common/metrics.py`
- artefatos deste artigo: `computational_results_20260402_222902/`
- figuras principais: `series_overview.png` e `weekly_profile.png`

0.12. Referencias

- Corporacion Favorita Grocery Sales Forecasting. Kaggle.
- Hyndman, R. J.; Athanasopoulos, G. Forecasting: Principles and Practice.
- Jaeger, H. The "echo state"approach to analysing and training recurrent neural networks.
- Lukosevicius, M.; Jaeger, H. Reservoir computing approaches to recurrent neural network training.